ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Data Type Design — Challenge Set Feedback

Part A — The Haskell datatype

```haskell
module Ledger (Transaction(..), Ledger, fee, gasCost, isValid,
               describe, totalFees, filterByAddress, txInRange, groupByKind) where

data Transaction
  = Transfer       { from :: String, to :: String, amount :: Double, timestamp :: Int }
  | ContractDeploy { owner :: String, bytecode :: String, gasLimit :: Int, timestamp :: Int }
  | ContractCall   { caller :: String, contract :: String, function :: String,
                     gasLimit :: Int, timestamp :: Int }
  deriving Show

type Ledger = [Transaction]

-- Transaction fee
fee :: Transaction -> Double
fee (Transfer       { amount   = a }) = 0.01 * a
fee (ContractDeploy { gasLimit = g }) = 0.000001 * fromIntegral g
fee (ContractCall   { gasLimit = g }) = 0.000001 * fromIntegral g

-- Estimated computational cost (in gas units)
gasCost :: Transaction -> Int
gasCost (Transfer       {})                     = 21000
gasCost (ContractDeploy { bytecode = bc })      = 32000 + 200 * length bc
gasCost (ContractCall   { function = f  })      = 50000 + 100 * length f

-- Validity check
isValid :: Transaction -> Bool
isValid t@(Transfer { amount = a, from = f, to = to' }) =
  a > 0 && f /= to'
isValid t@(ContractDeploy { bytecode = bc, gasLimit = gl }) =
  not (null bc) && gl >= gasCost t
isValid t@(ContractCall { function = fn, gasLimit = gl }) =
  not (null fn) && gl >= gasCost t
```

```haskell
-- Human-readable one-line description
describe :: Transaction -> String
describe (Transfer { from = f, to = t, amount = a, timestamp = ts }) =
  "Transfer: " ++ f ++ " -> " ++ t ++ " " ++ show a ++ " @" ++ show ts
describe (ContractDeploy { owner = o, gasLimit = gl, timestamp = ts }) =
  "Deploy: " ++ o ++ " deploys contract (" ++ show gl ++ " gas) @" ++ show ts
describe (ContractCall { caller = c, contract = ct, function = fn,
                         gasLimit = gl, timestamp = ts }) =
  "Call: " ++ c ++ " calls " ++ fn ++ " on " ++ ct
        ++ " (" ++ show gl ++ " gas) @" ++ show ts

-- Total fees across all transactions
totalFees :: Ledger -> Double
totalFees = sum . map fee

-- All transactions involving a given address
filterByAddress :: String -> Ledger -> Ledger
filterByAddress addr = filter (involves addr)
  where
    involves a (Transfer       { from   = f,  to       = t  }) = a == f || a == t
    involves a (ContractDeploy { owner  = o }) = a == o
    involves a (ContractCall   { caller = c,  contract = ct }) = a == c || a == ct

-- All transactions with timestamp in [lo, hi]
txInRange :: Int -> Int -> Ledger -> Ledger
txInRange lo hi = filter (\t -> timestamp t >= lo && timestamp t <= hi)

-- Partition into (transfers, deploys, calls)
groupByKind :: Ledger -> ([Transaction], [Transaction], [Transaction])
groupByKind = foldr step ([], [], [])
  where
    step t@(Transfer       {}) (ts, ds, cs) = (t:ts, ds,   cs)
    step t@(ContractDeploy {}) (ts, ds, cs) = (ts,   t:ds, cs)
    step t@(ContractCall   {}) (ts, ds, cs) = (ts,   ds,   t:cs)
```

Observations

A few things that came up in the draft on the board are worth flagging explicitly:

- **`fee` for contracts uses a non-integer multiplier.** `0.000001 * g` requires `fromIntegral g` because `g :: Int` and `0.000001 :: Double`. Haskell does not insert numeric coercions

automatically. The version on the board (`0.00001 * g`) was both a typo on the constant and a type error on the multiplication.

- **`groupByKind` as a `foldr`.** The accumulator is `([Transaction], [Transaction], [Transaction])`, and the step function dispatches on the variant. This is a clean illustration of `foldr` with a structured accumulator — worth lingering on if students were unsure about the type.

Answers to the discussion questions

Is the proposed datatype effective? Yes, but with one structural observation: the `timestamp` field appears identically in all three variants. The variant type *can* express the structure, but the repetition signals that timestamp is a *common* attribute and not really a per-variant property. A cleaner design would factor it out:

```
data Transaction = Tx { timestamp :: Int, payload :: TxPayload }

data TxPayload
  = Transfer       { from :: String, to :: String, amount :: Double }
  | ContractDeploy { owner :: String, bytecode :: String, gasLimit :: Int }
  | ContractCall   { caller :: String, contract :: String, function :: String, gasLimit :: Int }
```

We do not adopt this refactoring in the reference solution because it would force re-pattern-matching at every site — but it is the right answer to "is this datatype effective": it works, but it puts a common attribute inside the variants instead of around them.

`totalFees` higher-order function. `map` followed by `sum`, or equivalently `foldr (+) 0 . map fee`. The point-free `sum . map fee` is the most idiomatic. `filter` is the wrong tool here.there is no condition to filter on; every transaction contributes to the total.

`groupByKind` as a `foldr`. Yes, the accumulator is `([Transaction], [Transaction], [Transaction])` and the step function dispatches on the variant, prepending to the appropriate list. The result reverses traversal order; if order needs to be preserved, use `foldl` with a strictness annotation, or `foldr` with appending (slow), or partition explicitly.

Part B — OO design in Scala

Trait and case classes

```
sealed trait Transaction:
  def timestamp: Int
  def fee: Double
  def gasCost: Int
  def isValid: Boolean
  def describe: String

case class Transfer(from: String, to: String, amount: Double, timestamp: Int)
    extends Transaction:
  def fee: Double      = 0.01 * amount
  def gasCost: Int     = 21000
  def isValid: Boolean = amount > 0 && from != to
  def describe: String = s"Transfer: $from -> $to $amount @$timestamp"

case class ContractDeploy(owner: String, bytecode: String,
                          gasLimit: Int, timestamp: Int)
    extends Transaction:
  def fee: Double      = 0.000001 * gasLimit
  def gasCost: Int     = 32000 + 200 * bytecode.length
  def isValid: Boolean = bytecode.nonEmpty && gasLimit >= gasCost
  def describe: String = s"Deploy: $owner deploys contract ($gasLimit gas) @$timestamp"

case class ContractCall(caller: String, contract: String, function: String,
                        gasLimit: Int, timestamp: Int)
    extends Transaction:
  def fee: Double      = 0.000001 * gasLimit
  def gasCost: Int     = 50000 + 100 * function.length
  def isValid: Boolean = function.nonEmpty && gasLimit >= gasCost
  def describe: String = s"Call: $caller calls $function on $contract ($gasLimit gas) @$timestar
```

Package functions

```
package ledger

def totalFees(txs: List[Transaction]): Double =
  txs.map(_.fee).sum

def filterByAddress(addr: String, txs: List[Transaction]): List[Transaction] =
  txs.filter {
    case Transfer(f, t, _, _)            => addr == f || addr == t
    case ContractDeploy(o, _, _, _)      => addr == o
    case ContractCall(c, ct, _, _, _)    => addr == c || addr == ct
  }

def txInRange(lo: Int, hi: Int, txs: List[Transaction]): List[Transaction] =
  txs.filter(t => t.timestamp >= lo && t.timestamp <= hi)

def groupByKind(txs: List[Transaction])
    : (List[Transaction], List[Transaction], List[Transaction]) =
  txs.foldRight((List.empty[Transaction], List.empty[Transaction], List.empty[Transaction])):
    case (t: Transfer,       (ts, ds, cs)) => (t :: ts, ds, cs)
    case (t: ContractDeploy, (ts, ds, cs)) => (ts, t :: ds, cs)
    case (t: ContractCall,   (ts, ds, cs)) => (ts, ds, t :: cs)
```

Answers to the discussion questions

Adding an operation vs adding a variant. This is the *expression problem* in miniature:

- *Adding `isExpensive` (a new operation).* In Haskell, write a new top-level function with three clauses. Nothing else changes. In Scala OO, add a method to the trait *and* an implementation in each of the three case classes — three places to modify. **Haskell wins.**
- *Adding `TokenMint` (a new variant).* In Haskell, add a new constructor to the `data` declaration *and* a new clause to every function that pattern-matches on `Transaction` — every function. In Scala OO, add a new case class extending the trait, with implementations of all the methods. **Scala wins** (for self-contained case classes; package functions still need updating).

The two organisations make opposite operations cheap and the dual operation expensive. This is *the* recurring theme of the lecture, and the moment students recognise the symmetry is the most valuable takeaway.

Where does `isExpensive` go? It is a derived operation that depends only on `fee`, so it could be either. Three reasonable answers:

```
// (a) Method on the trait
sealed trait Transaction:
  def isExpensive: Boolean = fee > 100.0    // default implementation, no per-variant override r

// (b) Package function
def isExpensive(t: Transaction): Boolean = t.fee > 100.0

// (c) Extension method
extension (t: Transaction)
  def isExpensive: Boolean = t.fee > 100.0
```

Option (a) puts it with the other transaction-level operations and is the most discoverable. Option (b) keeps the trait minimal. Option (c) is in between. The default implementation in (a) is the killer feature: because `isExpensive` is fully derived from `fee`, no case class needs to override it. This is something Haskell cannot express directly — the equivalent there is just "define a top-level function".

`TokenMint` in Scala.

```
case class TokenMint(owner: String, tokenName: String, supply: Int,
                     gasLimit: Int, timestamp: Int)
    extends Transaction:
  def fee: Double      = 0.000001 * gasLimit
  def gasCost: Int     = 40000 + 50 * tokenName.length      // illustrative
  def isValid: Boolean = tokenName.nonEmpty && supply > 0 && gasLimit >= gasCost
  def describe: String = s"Mint: $owner mints $supply $tokenName ($gasLimit gas) @$timestamp"
```

What you have to modify: add the case class, and update `filterByAddress` and `groupByKind` (the package functions that pattern-match on the variants). The trait and the other case classes are untouched — this is exactly the OO advantage for adding variants. Note that `groupByKind`'s return type now needs a fourth list, so its signature also changes.

`TokenMint` in Haskell. Add a new constructor to `data Transaction`, and add a new clause to *every* function that pattern-matches: `fee`, `gasCost`, `isValid`, `describe`, `filterByAddress`, `groupByKind`. The compiler's non-exhaustive-pattern warnings are the safety net that tells you which functions still need updating.

Refining `groupByKind`'s return type. In Scala, yes:

```
def groupByKind(txs: List[Transaction])
    : (List[Transfer], List[ContractDeploy], List[ContractCall]) =
  txs.foldRight((List.empty[Transfer], List.empty[ContractDeploy], List.empty[ContractCall])):
    case (t: Transfer,       (ts, ds, cs)) => (t :: ts, ds, cs)
    case (t: ContractDeploy, (ts, ds, cs)) => (ts, t :: ds, cs)
    case (t: ContractCall,   (ts, ds, cs)) => (ts, ds, t :: cs)
```

This works because `Transfer`, `ContractDeploy`, and `ContractCall` are *types in their own right* — they are subtypes of `Transaction`, but each is also a class with its own type. Clients downstream can rely on `Transfer` having a `from` field, without re-checking the variant.

In Haskell, you cannot. The constructors `Transfer`, `ContractDeploy`, `ContractCall` are *not types*: they are inhabitants of the single type `Transaction`. There is no type `Transfer` that you could put inside `[Transfer]`. The closest thing in Haskell is GADTs or a phantom-tagged variant, both of which are heavy machinery for this purpose.

This is the most concrete illustration of the **constructor vs subtype** distinction the lecture set up:

- A *constructor* is a way to build a value of an existing type. `Transfer` produces a `Transaction`.
- A *subtype* is a type that is a refinement of another type. `Transfer <: Transaction` means `Transfer` is *itself* a type, and every `Transfer` is also a `Transaction`.

Haskell's variants are constructors only. Scala's case classes are *both* constructors *and* subtypes — when you write `case class Transfer(...) extends Transaction`, you get a constructor `Transfer(...)` *and* a type `Transfer`. That doubling is what lets the precise return type work.

Follow-up: the `Ledger` class

```
class Ledger(val txs: List[Transaction]):
  def totalFees: Double                                    = txs.map(_.fee).sum
  def filterByAddress(addr: String): Ledger                = Ledger(txs.filter(...))
  def txInRange(lo: Int, hi: Int): Ledger                  = Ledger(txs.filter(...))
  def groupByKind: (List[Transfer], List[ContractDeploy], List[ContractCall]) = ...
```

`Ledger(myTxs).totalFees` vs `totalFees(myTxs)`. The class buys you three things:

1. *Method-chaining.* `Ledger(myTxs).filterByAddress("alice").txInRange(1000, 1005).totalFees` reads as a pipeline; the package-function version requires nesting or temporary variables.
2. *Type identity.* `Ledger` is a distinct type from `List[Transaction]`, even though it wraps one. This prevents accidental confusion — you cannot pass a list where a ledger is expected.

The Haskell `type Ledger = [Transaction]` gets you none of these: a type alias is *transparent*, so `Ledger` and `[Transaction]` are interchangeable everywhere. The Haskell equivalent of the Scala class is `newtype Ledger = Ledger [Transaction]` — that introduces a genuinely distinct type at no runtime cost, and to operate on the underlying list you have to unwrap explicitly. It comes closer to the Scala class, though it still does not give you the OO method-call syntax.

Part C — The Elixir perspective

```elixir
defmodule Transfer do
  defstruct [:from, :to, :amount, :timestamp]
end

defmodule ContractDeploy do
  defstruct [:owner, :bytecode, :gas_limit, :timestamp]
end

defmodule ContractCall do
  defstruct [:caller, :contract, :function, :gas_limit, :timestamp]
end

defmodule Ledger do

  def fee(%Transfer{amount: a}),             do: 0.01 * a
  def fee(%ContractDeploy{gas_limit: g}),    do: 0.000001 * g
  def fee(%ContractCall{gas_limit: g}),      do: 0.000001 * g

  def gas_cost(%Transfer{}),                  do: 21_000
  def gas_cost(%ContractDeploy{bytecode: b}), do: 32_000 + 200 * String.length(b)
  def gas_cost(%ContractCall{function: f}),   do: 50_000 + 100 * String.length(f)

  def is_valid(%Transfer{amount: a, from: f, to: t}),
    do: a > 0 and f != t
  def is_valid(%ContractDeploy{bytecode: b, gas_limit: g} = tx),
    do: b != "" and g >= gas_cost(tx)
  def is_valid(%ContractCall{function: f, gas_limit: g} = tx),
    do: f != "" and g >= gas_cost(tx)

  def describe(%Transfer{from: f, to: t, amount: a, timestamp: ts}),
    do: "Transfer: #{f} -> #{t} #{a} @#{ts}"
  def describe(%ContractDeploy{owner: o, gas_limit: g, timestamp: ts}),
    do: "Deploy: #{o} deploys contract (#{g} gas) @#{ts}"
  def describe(%ContractCall{caller: c, contract: ct, function: f,
                             gas_limit: g, timestamp: ts}),
    do: "Call: #{c} calls #{f} on #{ct} (#{g} gas) @#{ts}"

  def total_fees(txs),                 do: txs |> Enum.map(&fee/1) |> Enum.sum()
  def filter_by_address(addr, txs),    do: Enum.filter(txs, &involves?(&1, addr))
  def tx_in_range(lo, hi, txs),        do: Enum.filter(txs, fn t -> t.timestamp >= lo and t.tir
```

```
  defp involves?(%Transfer{from: f, to: t}, a),         do: a == f or a == t
  defp involves?(%ContractDeploy{owner: o}, a),         do: a == o
  defp involves?(%ContractCall{caller: c, contract: ct}, a), do: a == c or a == ct

  def group_by_kind(txs) do
    Enum.reduce(txs, {[], [], []}, fn
      %Transfer{}       = t, {ts, ds, cs} -> {[t | ts], ds, cs}
      %ContractDeploy{} = t, {ts, ds, cs} -> {ts, [t | ds], cs}
      %ContractCall{}   = t, {ts, ds, cs} -> {ts, ds, [t | cs]}
    end)
  end
end
```

Answers to the discussion questions

Which style does the Elixir code resemble? The functional one. Operations are top-level functions in a module, dispatched on the struct via pattern matching — almost exactly the Haskell layout transposed into Elixir's syntax. There is no analogue of methods on the structs; behaviour lives in the module that operates on them. The structs themselves are just labelled records.

Passing a plain map to `fee/1`. What happens at runtime is a `FunctionClauseError`: the function expects values of types `%Transfer{}`, `%ContractDeploy{}`, or `%ContractCall{}`, and a plain map matches none of them. So *Elixir does catch the mistake* — but at run time, when the call actually happens, not at compile time. If the bug is in a code path that is rarely exercised, it can sit in the codebase for a long time before it fires.

In Haskell or Scala, the same mistake would be caught at compile time. In Haskell, a value of type `Map` cannot be passed where a `Transaction` is expected; the type mismatch is a compile error. In Scala, the same holds with the `Transaction` trait. This is the canonical illustration of the *static vs dynamic typing* trade-off: the dispatch logic is the same, the failure detection is not.

A precise return type for `group_by_kind`. Elixir does not have static types, so the answer is structural: there is no static return type at all, only the runtime tuple shape. What Elixir offers in place of a static refinement is `@spec`, an annotation read by Dialyzer (the optional static analysis tool):

```
@spec group_by_kind([Transfer.t() | ContractDeploy.t() | ContractCall.t()]) ::
        {[Transfer.t()], [ContractDeploy.t()], [ContractCall.t()]}
def group_by_kind(txs), do: ...
```

This *describes* the precise return type but does not enforce it: the code compiles and runs identically with or without the spec. Dialyzer, if invoked, will complain about callers that violate the spec. The same trade-off as before — static vs dynamic — at the level of refinement types: Scala enforces the precision at compile time, Elixir documents it and lets an external tool check it on demand.